Hashing methods comparison

8. Students in your class is 60, each contain 5 digit Roll number. If hashing table memory location is 97, find locations with (i) division method, (ii) Mid-square method & (iii) Folding method for the following Roll numbers. (99705, 99735 & 99759)

9. Suppose a Hash table contain 11 memory locations and a file contain 8 Records, where

Record	A	B	C	D	E	F	G	H
Hash address H(k)	4	8	2	11	4	11	5	1

Using linear probing method show how the records will appear in memory.

10. A Hash table contains 1000 slots, indexed from 0 to 999. The elements in the table have keys that range from 1 to 100000. The original hash function is Key MOD 1000. Which, if any, of the following collision resolution schemes would work correctly and why?

(i) Rehashing, with function = (Key+1) MOD 1000
(ii) Rehashing, with function = (Key+2) MOD 1000
(iii) Rehashing, with function = Key MOD 998
(iv) Rehashing, with function = (Key+3) MOD 1000

A Hash table is used to store employee records of EmployeeType whose key field is called IdNum. The key may range in value from 1000 to 9999. The original hash function is key MOD 100. There is a chain of records for each hash address. Write a procedure (function), which inserts an employee record into the appropriate chain.

solve this hashing questions

Let's solve the **hashing problems** in each of the three images you've uploaded:

📌 Q8. Hashing Roll Numbers using 3 Methods

Given:

- Hash table size = 97
- Roll numbers = `99705`, `99735`, `99759`

(i) Division Method

Formula: `Hash = Key % TableSize`

- `99705 % 97 = 64`
- `99735 % 97 = 94`
- `99759 % 97 = 18`

✅ **Locations**: `64, 94, 18`

(ii) Mid-Square Method

Steps:

1. Square the key.
2. Extract the middle digits (typically 2 or 3 depending on table size).
3. Apply mod 97.

- `99705² = 9941216025` → middle 3 digits: **160** → `160 % 97 = 63`
- `99735² = 9947092225` → middle 3 digits: **922** → `922 % 97 = 50`
- `99759² = 9951823281` → middle 3 digits: **823** → `823 % 97 = 48`

✅ **Locations**: `63, 50, 48`

(iii) Folding Method

Break the number and add segments:

- `99705`: → 997 + 05 = 1002 → `1002 % 97 = 31`
- `99735`: → 997 + 35 = 1032 → `1032 % 97 = 63`

- **99759**: → 997 + 59 = 1056 → **`1056 % 97 = 88`**

✅ **Locations**: **`31, 63, 88`**

📌 Q9. Linear Probing

Hash Table Size = 11
Records: A–H with hash addresses:

Record	A	B	C	D	E	F	G	H
Hash	4	8	2	11	4	11	5	1

Insert using linear probing:

1. A (4) → index 4 ✅
2. B (8) → index 8 ✅
3. C (2) → index 2 ✅
4. D (11) → index **0** (11 % 11 = 0) ✅
5. E (4) → index 4 ❌ occupied → try 5 ✅
6. F (11) → index 0 ❌ → 1 ❌ → 2 ❌ → 3 ✅
7. G (5) → index 5 ❌ → 6 ✅
8. H (1) → index 1 ❌ → 2 ❌ → 3 ❌ → 4 ❌ → 5 ❌ → 6 ❌ → 7 ✅

✅ **Final Table**:

Index	Record
0	D
1	
2	C
3	F
4	A
5	E
6	G
7	H
8	B
9	
10	

📌 Q10. Valid Collision Resolution

Original Hash: **`Key % 1000`**

(i) `(Key + 1) % 1000` ✅

- Will cover all 1000 slots eventually (step size = 1, co-prime to 1000)

(ii) `(Key + 2) % 1000` ❌

- Step = 2 → will only probe even addresses, missing half the table.

(iii) `Key % 998` ❌

- Different hash space; doesn't resolve collision, creates a different mapping.

(iv) `(Key + 3) % 1000` ✅

- Step = 3 and 3 is co-prime to 1000 → ensures full cycle through table

✅ **Correct Options**: (i) and (iv)

📌 Q11. Insert Employee into Hash Table with Chaining

Given:

- Key: **`IdNum`** (1000–9999)
- Hash function: **`IdNum % 100`**
- Chaining used

✅ Sample Python Function:

```
python

class Employee:
    def __init__(self, id_num, name):
        self.id_num = id_num
        self.name = name
        self.next = None

class HashTable:
    def __init__(self):
        self.table = [None] * 100  # size 100 for hash % 100

    def insert(self, id_num, name):
        index = id_num % 100
```

```
        new_emp = Employee(id_num, name)
        if self.table[index] is None:
            self.table[index] = new_emp
        else:
            current = self.table[index]
            while current.next:
                current = current.next
            current.next = new_emp
```

Let me know if you'd like the Python implementation in a different style (object-oriented, with search/delete, etc.) or explanation of another concept!

replace the code with diagram or simulation

Here's a **diagram-based simulation** to visually explain how **hashing with chaining** works for Q11 instead of showing code.

📌 Question Recap (Q11)

- Hash table has **100 slots** (`0–99`)
- Key is `IdNum` (1000–9999)
- Hash function: `IdNum % 100`
- Use **chaining** to handle collisions
- Insert records into the hash table based on this logic

🧠 Example Insertions

Let's insert 4 employee records:

Employee Name	IdNum	Hash Index (IdNum % 100)
Alice	1234	34
Bob	2234	34
Charlie	1777	77
Diana	9877	77

🧩 Hash Table with Chaining (Diagram)

We show only the affected indexes:

```
scss

Index    Records (Linked List Chain)
-------------------------------------
 34   →  [1234: Alice] → [2234: Bob]
 77   →  [1777: Charlie] → [9877: Diana]
```

Explanation:

- Alice (1234) and Bob (2234) both hash to index **34**, so they form a chain.
- Charlie (1777) and Diana (9877) hash to index **77**, forming another chain.

All other indexes from `0–99` are currently `NULL` or empty.

🔄 Simulation Summary

- **Hash Function**: Distributes keys into 100 "buckets" using modulus.
- **Chaining**: Each bucket uses a **linked list** to store multiple records in case of collision.
- In our example, indexes `34` and `77` have **chains** of employees.

Would you like an **interactive tool or animated simulation** to better understand this? I can guide you on how to build one in Excel or using an online hash visualizer.